



PDF Download
3447812.pdf
05 February 2026
Total Citations: 34
Total Downloads: 934

Latest updates: <https://dl.acm.org/doi/10.1145/3447812>

RESEARCH-ARTICLE

A Software/Hardware Co-Design of Crystals-Dilithium Signature Scheme

Published: 05 June 2021
Accepted: 01 January 2021
Revised: 01 August 2020
Received: 01 June 2020

[Citation in BibTeX format](#)

ZHEN ZHOU, Wuhan University, Wuhan, Hubei, China

DEBIAO HE, Wuhan University, Wuhan, Hubei, China

ZHE LIU, Nanjing University of Aeronautics and Astronautics, Nanjing, Jiangsu, China

MIN LUO, Wuhan University, Wuhan, Hubei, China

KKWANG RAYMOND CHOO, The University of Texas at San Antonio, San Antonio, TX, United States

Open Access Support provided by:

Wuhan University

Nanjing University of Aeronautics and Astronautics

The University of Texas at San Antonio

A Software/Hardware Co-Design of Crystals-Dilithium Signature Scheme

ZHEN ZHOU, School of Cyber Science and Engineering, Wuhan University, China and Cyberspace Security Research Center, Peng Cheng Laboratory, China

DEBIAO HE, School of Cyber Science and Engineering, Wuhan University, China

ZHE LIU, College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, China

MIN LUO, School of Cyber Science and Engineering, Wuhan University, China

KIM-KWANG RAYMOND CHOO, Department of Information Systems and Cyber Security, Department of Electrical and Computer Engineering, and Department of Computer Science, University of Texas at San Antonio, USA

As quantum computers become more affordable and commonplace, existing security systems that are based on classical cryptographic primitives, such as RSA and **Elliptic Curve Cryptography (ECC)**, will no longer be secure. Hence, there has been interest in designing **post-quantum cryptographic (PQC)** schemes, such as those based on **lattice-based cryptography (LBC)**. The potential of LBC schemes is evidenced by the number of such schemes passing the selection of NIST PQC Standardization Process Round-3. One such scheme is the Crystals-Dilithium signature scheme, which is based on the hard module-lattice problem. However, there is no efficient implementation of the Crystals-Dilithium signature scheme. Hence, in this article, we present a compact hardware architecture containing elaborate modular multiplication units using the Karatsuba algorithm along with smart generators of address sequence and twiddle factors for NTT, which can complete polynomial addition/multiplication with the parameter setting of Dilithium in a short clock period. Also, we propose a fast software/hardware co-design implementation on **Field Programmable Gate Array (FPGA)** for the Dilithium scheme with a tradeoff between speed and resource utilization. Our co-design implementation outperforms a pure C implementation on a Nios-II processor of the platform Altera DE2-115, in the sense that our implementation is 11.2 and 7.4 times faster for signature and

The work was supported by the Major Scientific and Technological Innovation Project of Shandong Province (No. 2020CXGC010115), the National Natural Science Foundation of China (Grants No.61972294, No. 61932016 and 61802180), the Special Project on Science and Technology Program of Hubei Province (Grant No. 2020AEA013), the Natural Science Foundation of Hubei Province (Grant No. 2020CFA052), and the Wuhan Municipal Science and Technology Project (Grant No. 2020010601012187). K.-K. R. Choo was supported by the Cloud Technology Endowed Professorship.

Authors' addresses: Z. Zhou, School of Cyber Science and Engineering, Wuhan University, 299 Bayi Road, Wuchang District, Wuhan 430072, China, Cyberspace Security Research Center, Peng Cheng Laboratory, Shenzhen 518000, China; email: zhouzhen0215@whu.edu.cn; D. He and M. Luo, School of Cyber Science and Engineering, Wuhan University, 299 Bayi Road, Wuchang District, Wuhan 430072, China; emails: hedebiao@163.com, mluo@whu.edu.cn; Z. Liu, College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, 169 Shengtai Xi Lu, Jiangning District Nanjing 210016, China; email: sduliuzhe@gmail.com; K.-K. R. Choo, Department of Information Systems and Cyber Security, Department of Electrical and Computer Engineering, and Department of Computer Science, University of Texas at San Antonio, 1 UTSA Cir, San Antonio, TX 78249, USA; email: raymond.choo@fulbrightmail.org.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Association for Computing Machinery.

1936-7406/2021/06-ART11 \$15.00

<https://doi.org/10.1145/3447812>

verification, respectively. In addition, we also achieve approximately 51% and 31% speed improvement for signature and verification, in comparison to the pure C implementation on processor ARM Cortex-A9 of ZYNQ-7020 platform.

CCS Concepts: • **Security and privacy** → **Digital signatures; Hardware security implementation;**

Additional Key Words and Phrases: Lattice-based cryptography(LBC), polynomial multiplication, Crystals-Dilithium, hardware implementation

ACM Reference format:

Zhen Zhou, Debiao He, Zhe Liu, Min Luo, and Kim-Kwang Raymond Choo. 2021. A Software/Hardware Co-Design of Crystals-Dilithium Signature Scheme. *ACM Trans. Reconfigurable Technol. Syst.* 14, 2, Article 11 (June 2021), 21 pages.

<https://doi.org/10.1145/3447812>

1 INTRODUCTION

Quantum computation is fast becoming a reality and more commonplace. While there are benefits associated with quantum computers (e.g., high-speed parallel processing and low energy consumption), there are also implications to existing security systems that rely on classical cryptographic primitives (e.g., RSA, Diffie-Hellman, and **Elliptic Curve Cryptography (ECC)**). For example, there are existing quantum algorithms that can efficiently solve the integer factorization problem and discrete logarithm problem [43].

Hence, there has been interest in **post-quantum cryptography (PQC)**, whose security is based on mathematical problems that are computationally hard to solve even using quantum algorithms. The U.S. **National Institute of Standards and Technology (NIST)** has been assessing potential post-quantum algorithms to be included in the next standard, where security level and implementation performance are two key evaluation criteria [3, 11]. Specifically, all submitted algorithms will be subject to a public, competition-like process, and as of April 2020 a total of 15 round-three candidates were selected, including seven finalists and eight alternate candidates.

Of the seven round-three finalists, five are based on **lattice-based cryptography (LBC)**. One key advantage of LBC is that the problems they rely on (e.g., **shortest vector problem (SVP)**, and **shortest independent vector problem (SIVP)**) are computationally hard to solve under worst-case hardness assumptions. In Ajtai's [2] work, the **small integer solution (SIS)** problem is used to build a family of potent one-way functions, where preimage computation means finding the length of shortest lattice vector up to a polynomial factor. The **learning with errors (LWE)** has also been proven to be computationally hard by a quantum reduction from SVP and SIVP [39], and hence, not surprisingly, LWE has been utilized in many cryptography schemes [35] and the relevant security reduction is tight.

However, cryptographers may prefer the ring-version and module-version of SIS and LWE, where the ideal lattice $\mathbb{Z}_q/f(x)$ is introduced. The preference is partly due to implementation efficiency, as an ideal lattice implementation [28] has additional algebraic structure (compared to the standard lattice). Hence, large-scale matrix multiplication can be replaced with polynomial multiplication, which reduces the cost of arithmetical operations significantly. Consequently, the corresponding lattice-based cryptosystems are more flexible and versatile.

In lattice-based schemes involving ideal lattice $\mathbb{Z}_q/f(x)$, polynomial multiplications are a significant proportion of arithmetic operations. Hence, there is renewed interest in fast polynomial multiplication [8, 10, 21, 30]. The reduction polynomial often used in ideal lattice is $f(x) = x^n + 1$ because polynomial multiplication can be accelerated using the **Number Theoretic Transform (NTT)** algorithm. NTT can be regarded as a variant of **Fast Fourier Transform (FFT)** [36],

and many FFT algorithms with different radices have been proposed since the 1960s. However, FFT based on radix-2,4,8 and split-radix are commonly used for implementation, especially the algorithms of Cooley-Tukey [12] and Gentleman-Sande [19] because of their simplicity. The case of NTT is no exception. The efficiency of NTT implementation largely depends on the designs of dataflow, concurrency, throughput, and modular multiplication on $\mathbb{Z}_q$.

Existing lattice-based schemes generally have a probabilistic inherent feature. Many of them involve rejection sampling because the implementations of probability distribution are usually highly correlated with a tradeoff between quality and efficiency. Also, it may be the most vulnerable part when subject to **side channel attacks (SCA)**. Uniform distribution is one of the easiest to implement. However, as a true random number is costly to generate, researchers usually use a pseudo random number generator to sample, like **linear feedback shift register (LFSR)** and hash function. A common method of generating pseudo random sequences is to select a seed as the input of an extendable-output hash function such as SHAKE128/SHAKE256 of the SHA-3 family, and splicing the output continuously. The family of SHA-3 is the current hash function standard selected by NIST.

Crystals-Dilithium is a signature scheme constructed on the “Fiat-Shamir with aborts” framework, and has been proven to be secure based on **Module-LWE (MLWE)** and **Module-SIS (MSIS)** problems in the random oracle model. Also, Dilithium is one of the three digital signatures finalists in round-three of the NIST PQC standardization Process. The public key size of Dilithium is small, compared to many other LBC schemes. This is because it uses and improves the technique of hints. The authors of Dilithium have also proposed a scheme (Dilithium-G), where the involved error distribution is Gaussian distribution. However, in order to simplify the sampling process, Dilithium is free of Gaussian distribution and only uniform distribution is used. Compared to the most efficient signature scheme without Gaussian sampling, its public key space cost is about 2.5 times less. The authors have implemented the Dilithium scheme on Intel Core-i7 4770k and an AVX2 optimization is also considered [15]. However, at the time of this research, there is no efficient FPGA implementation reported of the Crystals-Dilithium scheme.

In this article, we improve a classical hardware implementation for polynomial multiplication using the NTT algorithm and present a compact hardware architecture for polynomial vector operations, where both forward/inverse NTT and pointwise addition/multiplication with parameter setting of Dilithium can be completed in a short clock period. In our architecture, finely designed modular multiplication units and generators of both address access and twiddle factors are contained, which help reduce latency and improve efficiency. Based on our architecture, we propose FPGA implementations for the Crystals-Dilithium signature scheme on the platform Altera DE2-115 and Xilinx ZYNQ-7020.

A comparative summary between our implementation and existing classical implementations demonstrates that our architecture is suitable for embedded applications with limited area. Our evaluation findings also show that our implementation is faster by 11.2 times for signature and 7.4 times for verification, in comparison to the corresponding pure C implementation on a Nios-II processor of the platform DE2-115. As for ARM Cortex-A9 of ZYNQ-7020, the rates of speed improvement are about 51% and 31% for signature and verification, respectively.

The structure of our article is organized as follows. In Sections 2 and 3, we will introduce the related literature and relevant background knowledge. Section 4 presents our algorithms. The hardware architecture based on our algorithms and the corresponding modules are given in Sections 5 and 6, respectively. The performance evaluation and comparative summary are presented in Section 7. Finally, we conclude this article in the last section.

2 RELATED LITERATURE

Existing research on efficient implementation of LBC schemes based on ideal lattice mainly focuses on the components of polynomial multiplication with different algorithms like NTT and Schoolbook. Existing implementations of polynomial multiplication can be broadly categorized into those based on software and those based on hardware.

In 2014, Roy et al. [40] implemented a compact hardware crypto-processor in which the NTT algorithm was optimized in a way that inspired many subsequent architectures, including [38] and [34] which extended the technique to inverse transformation. Seiler et al. [42] provided a fast AVX2 optimized implementation of NTT by modifying the Montgomery reduction. Feng et al. [17] designed a multi-lane stockham NTT architecture with high parallelism, which uses many DSPs and can complete the transformations very quickly. Mert et al. [29] proposed a scalable NTT architecture, where the modulus length varies from 22 to 32 bits and it performs well in the **Fan-Vercauteren (FV)** homomorphic scheme. **Schoolbook polynomial multiplication (SPM)** is another common method for compact LBC scheme implementation, due to its low resource cost. Liu et al. [25] proposed an optimized implementation of SPM on Kintex-7.

Hash function is also widely studied, and the current international standard SHA-3 is described by NIST. Implementation of SHA-3 has a number of advantages because of its slice-oriented architecture [16]. Considering the optimization approaches of unrolling and pipelining, Wong et al. [45] presented an architecture with an optimal tradeoff between speed and resource cost. In the work of [32], a novel SHA-3 implementation involving the technology of ReRAM-based **in memory-computing (IMC)** was proposed, designed to achieve significant energy reduction.

In [33], the authors surveyed existing implementations of the entire lattice-based cryptosystems. At present, there is an efficient open source C++ library, FNLlib [1], which is continuously improved and can be used for polynomial arithmetic operations in LBC schemes based on ideal lattice. However, the corresponding hardware implementations are limited. The first hardware implementation of the PKE scheme based on RLWE was proposed by Göttert et al. [20] in 2012. In 2014, an implementation of the BLISS signature scheme on Xilinx Spartan-6 FPGA was introduced [37], where the CDT sampler was improved. For standard lattices, Howe et al. [22] presented an encryption engine based on the standard LWE problem, which incurs significantly less area resource on the lightweight Spartan-6 platform. Kuo et al. [24] proposed a hardware implementation of the NewHope scheme, which they claimed to be the first.

Recently, Basu et al. [7] presented a hardware-based comparison of many popular PQC schemes, including the Crystals-Dilithium (the focus in our work). However, the designs they discussed were generated by **High-Level Synthesis (HLS)** methods, which can be seen as heuristic but not practical hardware designs on many platforms. Banerjee et al. [6] provided a configurable processor for many LBC protocols, which can implement sampling, modular arithmetic, and polynomial multiplication efficiently. In the work of Alkim et al. [4], an optimized RISC-V implementation of the polynomial arithmetic used by the Kyber and NewHope scheme is proposed, designed to achieve better performance by extending optimizations for the ARM instruction set to RISC-V. Dang et al. [13] provided software/hardware co-design for three lattice-based key encapsulation mechanisms and demonstrated that such an approach outperforms purely software implementation. Fritzmann et al. [18] presented an intensive RISC-V architecture embedding a number of powerful accelerators to speed up LBC schemes, whose performance has been evaluated by ASIC implementation. In addition, Xin et al. [46] provided a domain-specific vector processor that supports parallel number theoretical transforms and vector sampling. Their architecture outperforms the performance in the approach [6] by an order of magnitude.

$$\begin{pmatrix} a_{11}(x) & a_{12}(x) & \cdots & a_{1l}(x) \\ a_{21}(x) & a_{22}(x) & \cdots & a_{2l}(x) \\ \vdots & \vdots & \ddots & \vdots \\ a_{k1}(x) & a_{k2}(x) & \cdots & a_{kl}(x) \end{pmatrix} \begin{pmatrix} s_1(x) \\ s_2(x) \\ \vdots \\ s_l(x) \end{pmatrix} + \begin{pmatrix} e_1(x) \\ e_2(x) \\ \vdots \\ e_k(x) \end{pmatrix} = \begin{pmatrix} b_1(x) \\ b_2(x) \\ \vdots \\ b_k(x) \end{pmatrix}$$

Fig. 1. Representation of module-learning with errors (MLWE).

3 PRELIMINARIES

In this section, we provide an overview of MLWE and MSIS problems, polynomial multiplication on ring $\mathbb{Z}_q/f(x)$, random sampling, and the Crystals-Dilithium scheme.

3.1 MIWE and MSIS

According to Regev [39], LWE is conjectured to be hard to solve, assuming the hardness of both SVP and SIVP. Thus, LWE can be used to create public-key cryptosystems, which are more secure than classical ones. A matrix $A \in \mathbb{Z}_q^{k \times n}$ with large dimensions, whose elements sampled from an identical distribution and independent of each other, results in significant savings for calculation. So rather than standard lattices, many lattice-based cryptosystems are based on ideal lattice, which enjoys additional algebra structure introduced in Lyubashevsky et al.'s work [26–28].

It is widely believed that the special structure of ideal lattices does not supply a shortcut [41]. However, in order to strike a balance between implementation efficiency and security level, MLWE was introduced in [9], which can be seen as a generalization of both LWE and RLWE.

The MLWE problem is defined as follows: Suppose $A(x)$ comes from $R_q^{k \times l}$ randomly and uniformly, $S(x)$ comes from R_q^l , and $E(x)$ comes from R_q^k according to a specific k -dimension (l -dimension) distribution, where k and l are usually much less than the degree n . The target is to find secret polynomial vector $S(x)$ given $A(x)$ and $B(x)$, where $B(x) = A(x)S(x) + E(x)$. l is called the module rank. This is the RLWE problem, when l is set as one. Figure 1 presents the matrix description of MLWE.

The MSIS problem is defined as follows: Given $A \in R_q^{k \times l}$ chosen from a specific distribution, the target is to find $z \in R_q^l$, which satisfies $Az = 0$ and $0 < \|z\|_\infty \leq \beta$. Here, β is a bound parameter. The norm restriction is necessary, otherwise the problem will be much easier by the Gaussian elimination method. Actually, in some sense SIS and LWE are dual problems to each other because $b = As + e$ is equivalent to $Bt = 0$, where $B = (A|I)$ and $t = (s^T, e^T - b^T)^T$. This transformable relationship holds analogously in the case of the module-version.

3.2 Polynomial Multiplication on $\mathbb{Z}_q/f(x)$

Practical fast polynomial multiplication over $\mathbb{R}_q$ has always been a topic of interest. The multiplication in the form of coefficients is computationally intensive because of the convolution calculation. In comparison, the counterpart in the form of function value needs just n pointwise multiplications, which is much easier. The transformation from expression of coefficients to values also implies quadratic computation complexity in general. However, selecting special points and polynomial $f(x)$, we can simplify the transformation computation complexity from $O(n^2)$ to $O(n \log n)$. This results in overall improvement for speed. The fast transformation algorithm is NTT.

Choosing $f(x) = x^n - 1$ and $1, w_n^1, \dots, w_n^{n-1}$ to be the n value points (here, w_k is the k -th primitive root on $\mathbb{Z}_q$), the correctness of NTT can be deduced from the fact that $c(x) = a(x)b(x) \in \mathbb{Z}_q/x^n - 1$ is equivalent to $c(w_n^i) = a(w_n^i)b(w_n^i)$, ($i = 0, 1, \dots, n-1$). The efficiency improvement is based on properties of primitive root w_k . An in-depth discussion of NTT is presented in [12, 19].

For the case of $f(x) = x^n + 1$, the only difference is that the set of value points should be $w_{2n}^1, w_{2n}^3, \dots, w_{2n}^{n-1}$. Note that we can reduce this case to the former case where $f(x) = x^n - 1$ with an additional operation called negative wrapping. To be exact, we can compute $\bar{a}(x) = a(w_{2n}x)$ and $\bar{b}(x) = b(w_{2n}x)$, then execute the NTT for $f(x) = x^n - 1$ to obtain $\bar{c}(x) = \bar{a}(x)\bar{b}(x) \in \mathbb{Z}_q/x^n - 1$. At last, we compute $c(x) = \bar{c}(w_{2n}^{-1}x)$, which satisfies $c(x) = a(x)b(x) \in \mathbb{Z}_q/x^n + 1$.

3.3 Sampling and Hash

Random sampling is one of the indispensable components of public-key cryptosystems. The security of cryptosystems depends heavily on the quality of random numbers, or specifically, the randomness and independence. The acquisition of true random source is costly, so in practice **pseudo-random number generator (PRNG)** is used. PRNG can efficiently generate long pseudo-random sequences resistant to statistical analysis.

A common PRNG method is to select a short random seed and then use a hash function to extend it. In the Dilithium scheme, the extended random bytes come from the states of Keccak. As the random numbers of many cryptosystems must be from particular ranges, the rejection sampling is usually involved. All the hash functions used in the Dilithium scheme are from the SHA-3 after some necessary modification.

3.4 Crystals-Dilithium Signature Scheme

The Crystals-Dilithium is one of the three round-three finalists signature algorithms in the NIST PQC Standardization Process. The security of Dilithium is based on the hardness of MLWE and MSIS, and the security level varies due to parameters k and l , while $\mathbb{R}_q$ is constant. This property eases implementation. The modulus $q = 8,380,417$ and degree of polynomial $n = 256$ do not change with the security level. We refer readers interested in other detailed descriptions and analysis to the supporting documentation submitted to NIST [15].

4 KEY ALGORITHMS

In this section, the detailed descriptions of the key algorithms (in pseudocode) we will be using are presented. We also present the relevant proofs and analysis.

4.1 Modular Multiplication on $\mathbb{Z}_q$

In Dilithium, modular multiplication is a fundamental arithmetic component throughout the entire scheme. In our hardware design, we use the Karatsuba algorithm [23] for the standard multiplication and the Montgomery algorithm [31] for modular reduction. For the multiplication $z = xy \bmod q$, the first input x comes from $[0, 2q - 1]$, while the second input y and output z are restricted to $[0, q - 1]$ ($q = 2^{23} - 2^{13} + 1$). Therefore, the bit length of two inputs and output is 24 at most.

We use Karatsuba in ternary form to compute ab , and each operand is divided into three 8-bit digits. Appropriate segmentation can help make full use of the DSP resources (or rather, the embedded multiplier elements) on FPGA platforms. The detailed analysis can be found in the next section. For Montgomery reduction, we choose $r = 2^{23}$, and the corresponding $q' = 2^{23} - 2^{13} - 1$ has a low-weighted binary form just as q . Here, $qq' = -1 \bmod r$ holds. So we can use addition/subtraction and shift operation to realize the multiplication with q and q' . The modular multiplication $z = \text{Mont}(x, y) = xyr^{-1} \bmod q$ is presented in Algorithm 1.

Note that by this modular multiplication, the product we obtain is the real product with a factor r^{-1} additionally. It seems that we need additional computation to eliminate it after every stage of butterfly operations. However, as one of the two multiplication operands is always the twiddle factor, we can replace w 's with wr 's (in the Montgomery domain) to avoid the need of domain

ALGORITHM 1: Modular multiplication on $\mathbb{Z}_q$.**Input:** $q = 2^{23} - 2^{13} + 1$, $q' = 2^{23} - 2^{13} - 1$, $r = 2^{23}$

$$x = (x_2 \ll 16) | (x_1 \ll 8) | x_0 \in [0, 2q - 1], \quad y = (y_2 \ll 16) | (y_1 \ll 8) | y_0 \in [0, q - 1]$$

(Here $|$ means bitwise OR operation)**Output:** $z = \text{Mont}(x, y) = xy r^{-1} \bmod q \in [0, q - 1]$ Below $i = 0, 1, 2$ and $(j, k, l) = (0, 1, 2), (1, 2, 0), (2, 0, 1)$ (In order to be concise and clear, here we use circular expressions. For instance, $tmpp_i = x_i y_i$ means $tmpp_0 = x_0 y_0, tmpp_1 = x_1 y_1, tmpp_2 = x_2 y_2$ while $tmpx_j = x_k + x_l$ means $tmpx_0 = x_1 + x_2, tmpx_1 = x_2 + x_0, tmpx_2 = x_0 + x_1$)

1: $tmpp_i = x_i y_i$

2: $tmpx_j = x_k + x_l$

3: $tmpy_j = y_k + y_l$

4: $tmpp_j = tmpp_k + tmpp_l$

5: $ttmp_i = tmpx_i tmpy_i$

6: $z_0 = (tmpp_2 \ll 32) | (tmpp_1 \ll 16) | tmpp_0$

7: $ttmpu_i = ttmp_i - tmpp_i$

8: $z_0 = z_0 + (ttmpu_1 \ll 16)$

9: $tmp = (ttmpu_0 \ll 16) + ttmpu_2$

10: $z_0 = z_0 + (tmp \ll 8) \quad / * \quad z_0 = xy \quad * /$

11: $z_1 = z_0 + (z_0 \ll 13) \bmod r$

12: $z_1 = r - z_1 \quad / * \quad z_1 = z_0 q' \bmod r \quad * /$

13: $carry = (z_0 \bmod r + z_1) \gg 23$

14: $z_2 = (z_0 \gg 23) + z_1$

15: $z_3 = z_2 - (z_1 \gg 10)$

16: $z_3 = z_3 + carry \quad / * \quad z_3 = (z_0 + qz_1) \gg 23 \quad * /$

17: $z = z_3 \text{ or } z_3 - q \text{ or } z_3 = z_3 - 2q \quad / * \text{ Adjust final range if it is larger than } q \quad * /$

transformation, which can improve efficiency. The on-the-fly generation of wr 's is also feasible by using the same modular multiplication and replacing the initial factors w 's with their corresponding wr 's. The next formula transformations (CT/GS butterfly operations and generation of twiddle factors) ensure that the domains of all the coefficients in the identical butterfly stages are kept in coherence with Montgomery multiplication but without the need of correction for the additional factor r^{-1} . The left are the original logical equations while the right are what we really implement in our article.

$$c/d = a \pm w \cdot b \quad \Rightarrow \quad c/d = a \pm wr \cdot b \cdot r^{-1} = a \pm \text{Mont}(wr, b),$$

$$d = w \cdot (a - b) \quad \Rightarrow \quad d = wr \cdot (a - b) \cdot r^{-1} = \text{Mont}(wr, a - b),$$

$$w_{new} = w_{old} \cdot w_{step} \quad \Rightarrow \quad w_{new} r = w_{old} r \cdot w_{step} r \cdot r^{-1} = \text{Mont}(w_{old} r, w_{step} r).$$

4.2 Forward/Inverse-NTT

One of the most time-consuming processes of Dilithium is the polynomial multiplication using the NTT algorithm. Algorithm 2 presents an iterative version of the NTT algorithm, introduced by Roy et al. [40]. They proposed a corresponding compact hardware accelerator for the NTT algorithm, which is a pretty classic architecture. In their work, both forward and inverse NTT are accelerated. But they asserted that one of their optimization techniques, where the negative-wrapped scaling and butterfly computation are integrated, only applies to the forward NTT. In

ALGORITHM 2: Iterative NTT.

Input: Coefficients $(a_0, a_1, \dots, a_{n-1})$ of $a(x) \in \mathbb{Z}_q/x^n + 1$ and n -th primitive root $w_n \in \mathbb{Z}_q$ of unity

Output: Values $A = (a(1), a(w_{2n}w_n), \dots, a(w_{2n}^{n-1}w_n^{n-1}))$

```

1:  $A \leftarrow \text{BitRev}(a)$ 
2: for  $m = 2$  to  $n$  by  $m = 2m$  do
3:    $w_m \leftarrow w_n^{n/m}$ 
4:    $w \leftarrow \text{squareroot}(w_m)$  // Negative wrapping.
5:   for  $j = 0$  to  $m/2 - 1$  do
6:     for  $k = 0$  to  $n - 1$  by  $m$  do
7:        $(t, u) \leftarrow (w \cdot A[k + j + m/2], A[k + j])$ 
8:        $(A[k + j], A[k + j + m/2]) \leftarrow (u + t, u - t)$ 
9:     end for
10:     $w \leftarrow w \cdot w_m$ 
11:  end for
12: end for

```

the work of [34, 38], the scaling and butterfly computation of inverse NTT transformation are integrated in the similar way used in forward transformation. These algorithms are just what we will use in our implementation, but we continue to improve this classical method in order to make it more compatible with our modular multiplication algorithm above. The pseudo-codes for both forward-NTT and inverse-NTT are shown in Algorithms 3 and 4, respectively.

The optimization we use in forward-NTT are described in lines 2–7 and 22 of Algorithm 3. One can easily follow that if we replace $w \leftarrow 1$ with $w \leftarrow w_{2n}$ in line 22 and delete lines 2–7, the function of the algorithm does not change at all. The reason why we separate the negative wrapping and butterfly operation of the last stage while keeping them integrated for other stages is presented in Section 5. The case of inverse-NTT is similar.

5 HARDWARE ARCHITECTURE

In this section, we present an overview of our hardware architecture and the techniques in our designs, as well as the corresponding analysis. The first subsection presents a list of the critical modules in our implementation while the following show detailed explanations and analysis in turn.

5.1 Hardware Modules

The core components of our hardware architecture are responsible for operations on polynomial vectors, or specifically, the forward/inverse-NTT and pointwise polynomial multiplication/addition. Moreover, we also consider involving Keccak into our hardware architecture. This is because we experimentally found that the proportion of implementation cost for hash function increases rapidly and becomes a new bottleneck after the polynomial operations are implemented on the hardware. Our hardware implementation for Keccak is somewhat straightforward, and hence we do not provide further details. On the platform ZYNQ-7020, we experimentally found that, as the software part is implemented on an ARM processor with high performance and has an independent clock drive with a high frequency, introducing Keccak into hardware exclusively does not result in considerable improvements. To achieve increased improvement, we optimize Keccak using the NEON vector instructions of Cortex-A9. The results of our profiling experiments and comparison with several existing works are presented in the following sections.

ALGORITHM 3: Iterative forward NTT.**Input:** $a(x) \in \mathbb{Z}_q[x]/x^n + 1$ and the k -th primitive root w_k of unity ($k = 2, 2^2, 2^3, \dots, n/2, n, 2n$)**Output:** $A = \text{NTT}(a)$.

```

1:  $A \leftarrow \text{BitRev}(a)$ 
2: for  $k = n/2$  to  $n - 1$  by 2 do
3:   Load from  $\text{MEM}[k/2]$  into  $(A_k, A_{k+1})$ 
4:    $(l_1, r_1) \leftarrow (A_k, A_{k+1})$ 
5:    $(A_k, A_{k+1}) \leftarrow (w_{2n}l_1, w_{2n}r_1)$ 
6:   Load from  $(A_k, A_{k+1})$  into  $\text{MEM}[k/2]$ 
7: end for
8: for  $m = 2$  to  $n/2$  by  $m = 2m$  do
9:    $w \leftarrow w_{2m}$ 
10:  for  $j = 0$  to  $m/2 - 1$  do
11:    for  $k = 0$  to  $n - 1$  by  $2m$  do
12:      Load from  $\text{MEM}[k/2 + j]$  into  $(A_{k+j}, A_{k+j+m/2})$ 
13:      Load from  $\text{MEM}[k/2 + j + m/2]$  into  $(A_{k+j+m}, A_{k+j+3m/2})$ 
14:       $(l_1, r_1, l_2, r_2) \leftarrow (A_{k+j}, A_{k+j+m/2}, A_{k+j+m}, A_{k+j+3m/2})$ 
15:       $(A_{k+j}, A_{k+j+m/2}, A_{k+j+m}, A_{k+j+3m/2}) \leftarrow (l_1 + wr_1, l_1 - wr_1, l_2 + wr_2, l_2 - wr_2)$ 
16:      Load from  $(A_{k+j}, A_{k+j+m})$  into  $\text{MEM}[k/2 + j]$ 
17:      Load from  $(A_{k+j+m/2}, A_{k+j+3m/2})$  into  $\text{MEM}[k/2 + j + m/2]$ 
18:    end for
19:     $w \leftarrow w \cdot w_m$ 
20:  end for
21: end for
22:  $w \leftarrow 1$ 
23: for  $j = 0$  to  $n/2 - 1$  do
24:   Load from  $\text{MEM}[j]$  into  $(A_j, A_{j+n/2})$ 
25:    $(l_1, r_1) \leftarrow (A_j, A_{j+n/2})$ 
26:    $(A_j, A_{j+n/2}) \leftarrow (l_1 + wr_1, l_1 - wr_1)$ 
27:   Load from  $(A_j, A_{j+n/2})$  into  $\text{MEM}[j]$ 
28:    $w \leftarrow w \cdot w_n$ 
29: end for

```

The main hardware modules in our architecture are as follows: *Signalctrl*, *Addrctrl*, *Twiddlectrl*, *Alu* and a dual-port RAM with the size of $16 \times 128 \times 64 = 131,072$ bits. The data ports of RAM are 64-bit, which can load two coefficients (high 32 bits and low 32 bits). It can store 15 polynomials simultaneously, and the 0-th corresponds to the state of reset and is never used. Here, we describe the functions for the other four logical modules. The interface and FSMs are introduced in Section 6.

- **Addrctrl:** It provides addresses of read/write and enable signals *ren/wen* to RAM. The done signal is also generated by this module.
- **Alu:** It is responsible for butterfly computation, pointwise multiplication/addition, and the control of dataflow.
- **Twiddlectrl:** It generates the sequences of twiddle factors for forward-NTT and inverse-NTT.
- **Signalctrl:** It makes the necessary clock delays for signals to keep all the modules in step.

ALGORITHM 4: Iterative backward NTT.

Input: The inverse n^{-1} of n and the k -th primitive root w_k of unity. ($k = 2, 2^2, 2^3, \dots, n/2, n, 2n$) and $A = NTT(a)$ where $a(x) \in \mathbb{Z}_q[x]/x^n + 1$

(Note that all the w_k 's used here are the inverse elements of those used in forward-NTT.)

Output: $a(x) \in \mathbb{Z}_q[x] = \text{invNTT}(A)$.

```

1: for  $m = n/2$  to 2 by  $m = m/2$  do
2:   if  $m = n/2$  then
3:      $(w_l, w_h) \leftarrow (1, w_4)$ 
4:   else
5:      $(w_l, w_h) \leftarrow (w_{4m}, w_{4m} \cdot w_4)$ 
6:   end if
7:   for  $j = 0$  to  $m/2 - 1$  do
8:     for  $k = 0$  to  $n - 1$  by  $2m$  do
9:       Load from  $MEM[k/2 + j]$  into  $(A_{k+j}, A_{k+j+m})$ 
10:      Load from  $MEM[k/2 + j + m/2]$  into  $(A_{k+j+m/2}, A_{k+j+3m/2})$ 
11:       $(l_1, r_1, l_2, r_2) \leftarrow (A_{k+j}, A_{k+j+m}, A_{k+j+m/2}, A_{k+j+3m/2})$ 
12:       $(A_{k+j}, A_{k+j+m}, A_{k+j+m/2}, A_{k+j+3m/2}) \leftarrow (l_1 + r_1, w_l(l_1 - r_1), l_2 + r_2, w_h(l_2 - r_2))$ 
13:      Load from  $(A_{k+j}, A_{k+j+m/2})$  into  $MEM[k/2 + j]$ 
14:      Load from  $(A_{k+j+m}, A_{k+j+3m/2})$  into  $MEM[k/2 + j + m/2]$ 
15:    end for
16:     $(w_l, w_h) \leftarrow (w_l \cdot w_{2m}, w_h \cdot w_{2m})$ 
17:  end for
18: end for
19: for  $j = 0$  to  $n/2 - 1$  do
20:   Load from  $MEM[j]$  into  $(A_{2j}, A_{2j+1})$ 
21:    $(l_1, r_1) \leftarrow (A_{2j}, A_{2j+1})$ 
22:    $(A_{2j}, A_{2j+1}) \leftarrow (l_1 + r_1, w_4(l_1 - r_1))$ 
23:   Load from  $(A_{2j}, A_{2j+1})$  into  $MEM[j]$ 
24: end for
25: for  $k = 0$  to  $n/2 - 1$  by 2 do
26:   Load from  $MEM[k/2]$  into  $(A_k, A_{k+1})$ 
27:    $(l_1, r_1) \leftarrow (A_k, A_{k+1})$ 
28:    $(A_k, A_{k+1}) \leftarrow (n^{-1}l_1, n^{-1}r_1)$ 
29:   Load from  $(A_k, A_{k+1})$  into  $MEM[k/2]$ 
30: end for
31: for  $k = n/2$  to  $n - 1$  by 2 do
32:   Load from  $MEM[k/2]$  into  $(A_k, A_{k+1})$ 
33:    $(l_1, r_1) \leftarrow (A_k, A_{k+1})$ 
34:    $(A_k, A_{k+1}) \leftarrow (n^{-1}w_{2n}l_1, n^{-1}w_{2n}r_1)$ 
35:   Load from  $(A_k, A_{k+1})$  into  $MEM[k/2]$ 
36: end for
37:  $a \leftarrow \text{BITREV}(A)$ 

```

5.2 Pipeline of Modular Multiplication

Pipelining is a common method to shorten the critical path and increase clock frequency for hardware implementation. In our design, we use a 16-stage pipeline for the modular multiplier, where standard multiplication by the Karatsuba algorithm takes eight clocks and modular reduction by the Montgomery algorithm takes seven clocks. The last 16-th clock is just a register buffer. Table 1 shows the correspondence among the clock number, each step, and the tasks in Algorithm 1. As we previously discussed, the operands were segmented into three 8-bit digits using the

Table 1. Tasks of Every Clock in Modular Multiplication

Clocks	Steps	Workload
clk 1	input	input the operands
clk 2	lines 1–3	three 8-bit mul s , six 8-bit add s
clk 3	lines 4–6	three 9-bit mul s , three 16-bit add s
clk 4	line 7	three 18-bit sub s
clk 5	lines 8–9	one 29-bit add , one 18-bit add
clk 6	line 10	one 27-bit add
clk 7	line 10	one 14-bit add
clk 8	line 10	output the real product
clk 9	line 11	one 10-bit add
clk 10	line 12	one 23-bit sub
clk 11	lines 13–14	two 23-bit add s
clk 12	line 15	one 24-bit sub
clk 13	line 16	one 25-bit add
clk 14	line 17	one 25-bit sub , one 24-bit sub
clk 15	line 17	one 4-mux
clk 16	wait	null buffer

Operations in identical clock period are independent.

Karatsuba algorithm. In addition to the improvement brought by reduction in computational complexity using Karatsuba, another benefit is that multiplication of short-bit digits can reduce usage of computing resources to some extent. For instance, on the DE2-115 platform, the smallest multiplier is 9-bit; if the intermediate operands are longer than it, even for just one or two bits, additional multipliers will be called while the higher bits are never used. This implies a larger area.

In view of the generation and use of twiddle factors, it is necessary to choose a power of 2 as the clock number of pipeline considering the resolution from counter signals to address signals. We choose 16-clocks, which allows us to achieve a satisfactory clock frequency. For the multiplication between twiddle factors and butterfly operands, it is feasible to omit the 16-th stage to minimize trivial area cost. The guideline of pipeline design is minimal register usage, so the task allocation for every cycle may be uneven. However, it does not matter because the bottleneck of critical path and frequency lies on other modules instead of this part.

5.3 Scheduling of Twiddle Factors

Both butterfly operation and the on-the-fly generation of twiddle factors involve modular multiplication. To ensure that the butterfly operation is performed once per clock cycle, we need to assign several registers for the twiddle factors to work and renew in rotation to deal with the cycle delays of the pipeline. In Roy's work [40], a look-up table is used to select the corresponding w of the current cycle, while we use the method of register rotation to replace it so that many hardware selectors can be removed and the critical path is shortened. This method works because the clocks of every NTT stage are constant (128 clocks in Dilithium). Also, the buffer of Dout-low in [40] is not needed.

However, the additional cost is that we must store the initial values of all twiddle factor sequences. The used w 's of different NTT stages are completely different from each other. Actually, there is $k_m = 2^m l$ for all the factors used in the m -th NTT stage, where l is odd and k_m is the exponent of base w_{2n} . Therefore, we need to store $15 + 7 + 3 + 1 = 26$ factors more for forward-NTT and $14 + 6 + 2 = 22$ factors more for inverse-NTT, which is a considerable cost. In [40], a technique of parallelization and factors reuse is involved so the factors necessary to store are less.

Table 2. The Twiddle Factors Needed to be Prestored in Different Designs.

Stage	Twiddle Factors*
0	128
1	64 192
2	32 96 160 224
3	16
4	8 24
5	4 12 20 28
6	2 6 10 14 18 22 26 30
7 (non-optimized)	1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31
7 (optimized)	<i>1 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30</i>

*The bold numbers correspond to w 's prestored in [40], and non-bold are additional factors needed in our pipeline design. If the seventh stage are optimized, the numbers in the gray box can be replaced with the italic set, which can save storage space.

However, in this work, we do not use it considering the universality of architecture. In our design, we split the negative wrapping from the last stage of forward-NTT butterfly computation and the first stage for inverse-NTT. Thus, only 11 and 10 additional factors are needed for forward-NTT and inverse-NTT, respectively.

Table 2 describes the exponents of the used twiddle factors both in [40] and our pipeline architecture of optimized/non-optimized versions for forward NTT. The numbers in bold correspond to w 's needed in [40], and the others are additional factors needed in our pipeline design. The factors in the gray box are not necessary if we optimize the seventh stage. Moreover, non-bold numbers in stages 0,1,2 are also prestored, because we do not generate them on-the-fly for simplicity. The case of inverse-NTT is similar, and therefore we omit the description.

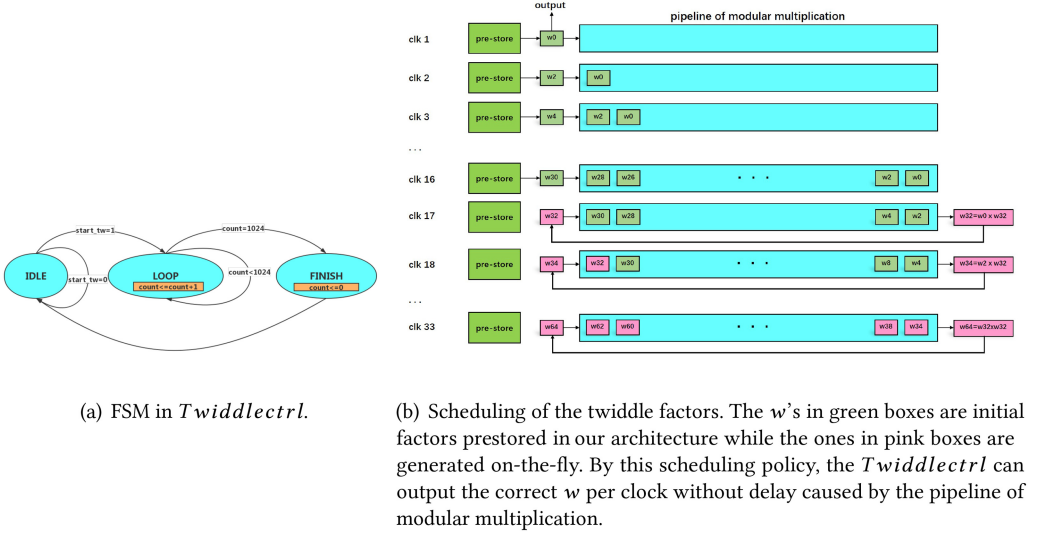
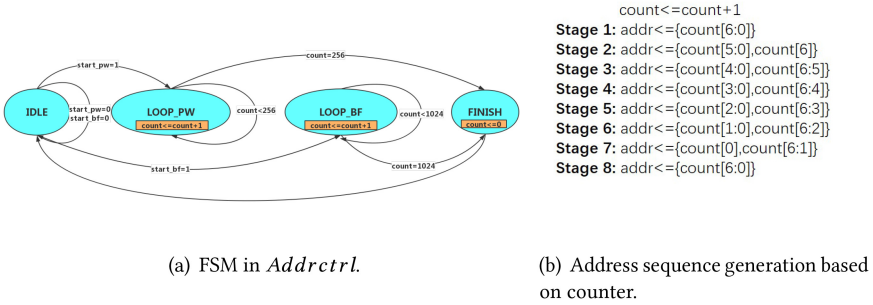
The separated wrapping operation for inverse-NTT can be merged into the last pointwise multiplications with r^2n^{-1} , while one more stage of pointwise multiplications (128 clocks) for forward-NTT is needed. However, this helps to correct the ranges of some variables generated from sampling, which may be larger than q .

5.4 Generation of Address Sequence

The polynomials expected to execute the NTT algorithm are stored in the dual-port RAM. The whole algorithm can be regarded as repeatedly reading the data from the specific address, processing it using the arithmetic module, and writing it back. We use the access scheme in [40] but the delay between read signal and write signal needs to be modified according to the pipeline constraint in our own architecture. A core idea of this algorithm is continuously processing the input data pairs of butterfly operation with the same twiddle factors in every NTT stage while jumping with the generation order of twiddle factors. Fortunately, the transformation law can be easily implemented by a hardware counter with trivial bits adjustment, which has been presented in the next section and Figure 3(b). In addition, for both forward/inverse NTT and pointwise multiplication/addition, the done signal is produced by the address module because it can directly and conveniently reflect whether the corresponding algorithms have been finished.

6 FSMS AND INTERFACE RESOLUTION

In our hardware design, by removing as many irregular clock delays in modules as possible, we successfully extract one module *Signalctrl*. This results in us having the start signal for different

Fig. 2. Flowchart of *Twiddlectrl*.Fig. 3. Flowchart of *Addrctrl*.

modules with correct clock delays just at the beginning to ensure all FSMs work orderly. Three FSMs are used and just one FSM for each of *Addrctrl*, *Alu*, and *Twiddlectrl*. Figures 2–4 show the flowcharts of all FSMs and the core details for the process in phase “LOOP.” One can observe that all FSMs only require one external start signal, which drives them from the idle state to the working state. After the tasks are finished, all of them go into the finish state adaptively. This property enables us to use *Signalctrl* to control all FSMs at the same time.

In Figure 2(b), we show a simple scheduling diagram of the twiddle factors in the last stage of forward-NTT in order to help understand. At the beginning, the factors prestored are both outputted during initial clock cycles and inputted into our modular multiplication unit. This is because *Twiddlectrl* is expected to provide a new factor every clock but the renewal involving multiplication cannot be finished so fast. When the factors get through the whole pipeline, they are reintroduced for both the current output and next corresponding factor generation. This circulation doesn’t end until the current stage of NTT is finished. The cases of other stages are similar. Figure 3(b) explains how we use a simple counter with nothing but position adjustment to implement the variable address step size for different NTT stages. As mentioned above, the module *Alu* finishes the butterfly and pointwise operations as well as the regrouping of coefficients. In

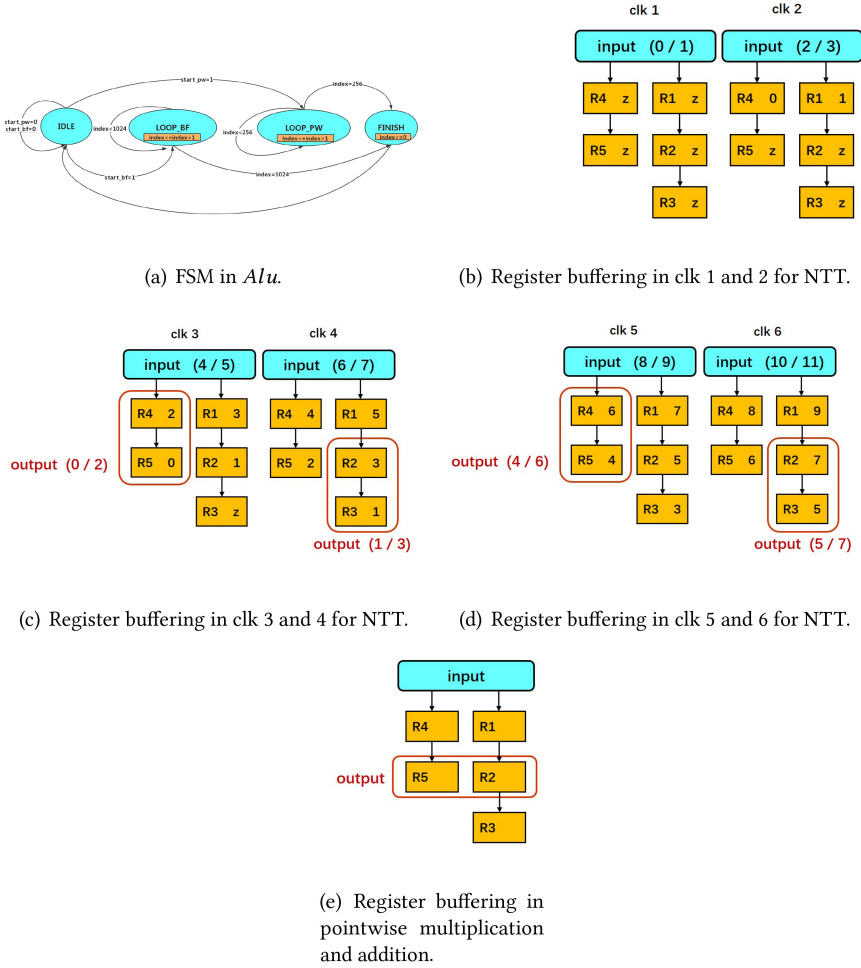
Fig. 4. Flowchart of *Alu*.

Figure 4(b)–(d), we demonstrate the registers buffering of the first six clock periods which can almost encapsulate all cases. As for the pointwise multiplication and addition, we just use the constant buffering in Figure 4(e).

Figure 5 presents the interface connection diagram of our architecture. The RAM in our architecture is dual-port and the size of width and depth are 64 bits and $16 \times 128 = 2,048$. One address corresponds to two coefficients (high 32 bits and low 32 bits), which means 128 addresses are needed for loading a whole polynomial of degree 255. Our RAM can store 15 polynomials simultaneously since the 0-th corresponds to the reset and is never used. The RAM is connected to both the external bus and the internal components. As we use NTT-CT for the forward and NTT-GS for the inverse, the initial input and final output coefficients of polynomials should be of the bit-reverse order. The reset states of all the RAM positions are zero polynomials. Polynomials to be processed are loaded into RAM through the bus directly. After the instructions mentioned below are resolved, the hardware modules start to read data from RAM and write the processed data back to RAM. At last, the resultant polynomials are loaded back into software workplace from RAM also through the bus. There are six instructions for our hardware architecture (the instructions for Keccak are omitted), as follows:

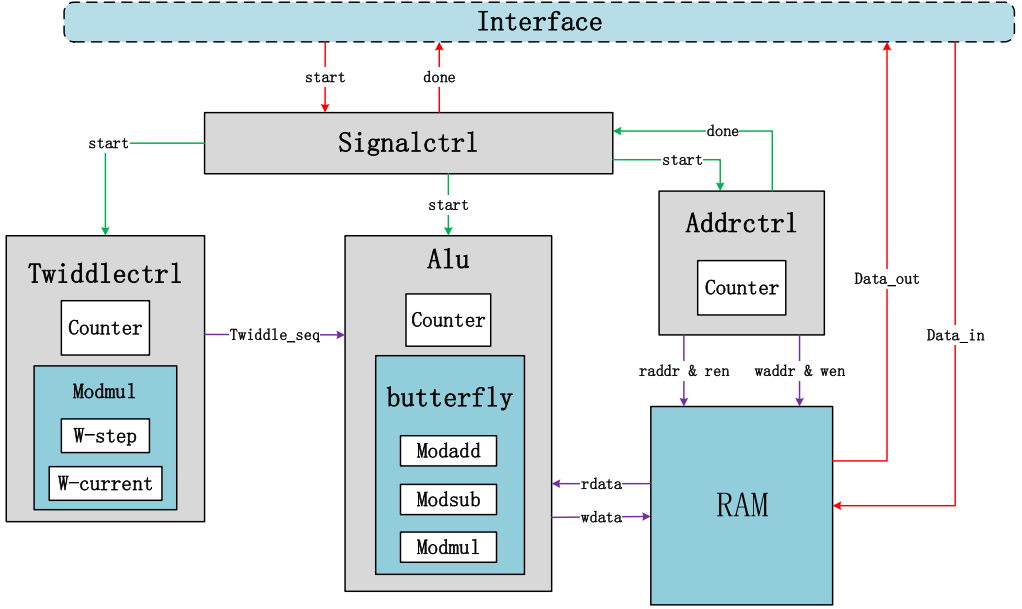


Fig. 5. Interface connection diagram of our architecture. Each of the three modules contains a counter. The colored arrows describe the signal flows and we used colors for clarity.

- **Data_in** ($int\ k, poly\ a$): Load a polynomial a from software workplace into the k -th position of RAM in our architecture.
- **Data_out** ($int\ k$): Load a polynomial from k -th position of RAM in our architecture into a specified position of software workplace.
- **NTT_CT** ($int\ k$): Take forward-NTT of a polynomial in the k -th position of RAM in our architecture (without negative wrapping of the last stage).
- **NTT_GS** ($int\ k$): Take inverse-NTT of a polynomial in the k -th position of RAM in our architecture (without negative wrapping of the first stage and the multiplication with inverse of n).
- **NTT_PW** ($int\ i, int\ j, int\ k$): Take pointwise multiplication of two polynomials in the i, j -th positions of RAM and store the product in the k -th positions of RAM in our architecture. (Notice that this instruction can also finish the missing parts of **NTT_CT** and **NTT_GS**.)
- **NTT_PA** ($int\ i, int\ j, int\ k$): Take pointwise addition of two polynomials in the i, j -th positions of RAM and store the sum in the k -th positions of RAM in our architecture.

In addition, we also present an example of how to execute the inner product of two four-dimensional polynomial vectors using our architecture in Algorithm 5. The cases of single polynomial multiplication and product between matrix and vector are similar. In our implementation of the Dilithium scheme, all the polynomial addition/multiplication operations are finished in hardware.

7 PERFORMANCE EVALUATION

In this section, we present a detailed explanation of how we conduct experiments, necessary analytical work, and the comparison with some classic existing designs.

ALGORITHM 5: An example of how we use our architecture to finish the inner product of two polynomial vectors.

Input: The polynomials $a_1(x), a_2(x), a_3(x), a_4(x), b_1(x), b_2(x), b_3(x), b_4(x) \in \mathbb{Z}_q[x]/x^n + 1$ from software workplace.

Output: $P(x) = a_1(x)b_1(x) + a_2(x)b_2(x) + a_3(x)b_3(x) + a_4(x)b_4(x)$ in software workplace.

```

1: Set  $c_0(x) = \sum_{i=0}^{127} r \cdot x^i + \sum_{i=128}^{255} r w_{512} \cdot x^i$ ,  $c_1(x) = \sum_{i=0}^{127} r^2 n^{-1} \cdot x^i + \sum_{i=128}^{255} r^2 w_{512}^{-1} n^{-1}$ 
   //Here  $r = 2^{23}$ ,  $w_{512}$  is the 512-th primitive root of unity.
2: Data_in(1,  $c_0(x)$ )
3: Data_in(2,  $c_1(x)$ )
   //These two constant polynomials need to be loaded only once during the whole scheme.
4: for  $s = 1$  to 4 do
5:   Data_in(2 +  $s$ ,  $a_s(x)$ )
6:   Data_in(6 +  $s$ ,  $b_s(x)$ )
7: end for
8: for  $t = 1$  to 8 do
9:   NTT_PW(1, 2 +  $t$ , 2 +  $t$ )
10:  NTT_CT(2 +  $t$ )
11: end for
12: for  $u = 1$  to 4 do
13:  NTT_PW(2 +  $u$ , 6 +  $u$ , 10 +  $u$ )
14:  NTT_PA(15, 10 +  $u$ , 15)
15: end for
16: NTT_GS(15)
17: NTT_PW(15, 2, 15)
18:  $P(x) = \text{Data\_out}(15)$ 

```

7.1 Experiments

In order to assess the performance of our design, we run our cooperative implementation and the corresponding pure software implementation on the same FPGA platforms with the same embedded soft-core processors. This will allow us to identify any performance improvements brought by offloading operations to hardware. We present the performance comparison results of two aspects, polynomial multiplication by NTT and the whole Dilithium scheme, with some existing implementations.

The security level we choose is the recommended version (equivalent to a secure symmetric encryption with 128-bits key), where $k = 5$, $l = 4$. The reference software code we use is from the repository *PQClean* on github [44], which is free of many external libraries that embedded systems might not support. The cooperative implementation of our design is modified based on this project.

We choose verilog as our hardware description language and the reference project from [44] is written in C. The FPGA tools we use are Quartus 18.1 for Cyclone IV and Vivado 2019.2 for Zynq-7020. For our Cyclone IV environment, the platform we use is the board Altera DE2-115 with the device Cyclone IV EP4CE115F29C7. Software implementation is finished on the embedded Nios II processor. The communication between processor and hardware we use is the common avalon bus protocol. The clock frequency we set is 100 MHz for all the architectures. For the second environment Zynq-7020, the platform involved is the board Xilinx AX7020 with device Zynq 7000 Soc XC7Z020-2CLG400I. Software implementation is finished on the embedded ARM Cortex-A9 processor. Communication used between PS and PL is AXI4-Lite protocol. The NEON

Table 3. Comparison between the Results of NTT (Degree $n = 256$) in Our Architecture and Several Classical Designs

Design	Modulus	Device	LUT/Slice	DSP	BRAM(18k/9k)	Freq./Cyc.
Our design	8,380,417	Cyclone IV	2921/-	12	-/15	182/1,170
	8,380,417	Xilinx Artix-7	2044/-	16	6/-	216/1,170
Design in [40]	7,681	Xilinx Virtex-7	1349/-	1	2/-	313/ $\approx$ 1,215
	12,289		1536/-	1	3/-	278/ $\approx$ 1,215
Design in [17]	1,049,089	Xilinx Spatan-6	-/14k	128	1/-	-/220
Design in [14]	–	Xilinx Spatan-6	533/-	1	2/-	233/4,066
Design in [6]	7,681	TSMC 40 nm LP CMOS	-/-	-	-/-	72/1,289

vector instructions are also involved in our experiment on the Zynq platform. There are dependent clocks for the processor and hardware modules. The clock frequency for Cortex-A9 is the default 533.3 MHz while 100 MHz for hardware architecture. More detailed information can be found in the following.

The correctness of our implementation is verified with the help of the official implementation submitted to NIST. We input the 10,000 signatures randomly generated in our implementation into the verification phase of [5] and believe our signature implementation is correct if the outputs are all valid. In addition, for the correctness of verification implementation, firstly, 10,000 standard signatures generated by [5] are introduced into our implementation where the outputs are all expected to be valid. Then 10,000 randomly generated strings are tested, which are expected to be determined invalid to prove our implementation is not trivial constant. As for experiment timing measurement, the metric we use is the average running time of 5,000 executions.

7.2 Polynomial Operations: A Comparative Summary

Table 3 is a comparative summary of the NTT algorithm costs between our architecture and classical architectures or processors. The performance indicators we consider are maximum frequency, clock cycles, and utilization of LUT, DSP, and BRAM.

To our knowledge, the parameter $q = 8,380,417$ only appears in the Dilithium signature scheme, while the modulus in most crypto-processors proposed have a short bit-length. As the polynomial degree $n = 256$ is pretty common and there are many corresponding existing works, we fix it in our comparison for fairness to some extent. In the design of [40], the computation of twiddle factors is separated from the butterfly operations, and the length of parameter $q = 7,681/12,289$ is shorter than the size of DSP on the platform they used. So they only need one DSP for multiplication operations. However, in our architecture, the computation of twiddle factors and butterfly operations are simultaneous, so we need two sets of standard multiplication components. Also, the longer modulus means more DSP utilization. The case for LUTs consumption is similar. However, we still achieve about 45 cycles reduction with an ideal frequency around 200 MHz. In the work of [17], the lengths of modulus enjoy a comparable bit length with Dilithium, but the NTT algorithm they used is Multilane Stockham, which takes full advantage of parallelism, therefore, about 5–10 times as much LUTs and DSPs but only about 20% cycles are required compared with ours. In the work of [14], only about a quarter of LUTs are consumed at the cost of about four times as many cycles compared with our results. In [6], the results of both NTT and the entire Dilithium scheme are provided on the cryptoprocessor named Sapphire. Compared with this work, we also achieve a saving of more than 100 cycles. The corresponding comparison of the whole scheme is included in the following. In the aspect of memory, the considerable BRAM resource cost of our design is

Table 4. Signature Performance: A Comparative Summary

Signature	Environment	LUT	Time(ms)	Time Saving(%) ^{#1}	Speed-up ^{#2}
Our implementation ¹	Altera Cyclone IV EP4CE115F29C7	987	4.40×10^3	0	1
Our implementation ²		3908	2.03×10^3	53.8	2.16
Our implementation ³		8568	3.91×10^2	91.1	11.2
Our implementation ^I	Xilinx Zynq 7000 Soc XC7Z020-2CLG400I	576	2.58×10^1	0	1
Our implementation ^{II}		2620	1.56×10^1	39.5	1.65
Our implementation ^{III}		6253	1.51×10^1	41.4	1.70
Our implementation ^{IV}		2620	1.26×10^1	51.1	2.04
Implementation in [6]	TSMC 40nm LP CMOS	-	8.82	/	/
Implementation in [7] ^a	Xilinx Artix-7 /*	133461	6.1	0	1
Implementation in [7] ^b		584742	1.85×10^{-1}	96.9	32.9
Implementation in [7] ^c		1327355	1.55	74.5	3.93

due to the fact that the Dilithium scheme is based on MLWE/MSIS. Therefore, the architecture is expected to load several polynomials simultaneously unless tolerating communication overhead between hardware and processor. To sum up, for the NTT algorithm, our architecture achieves a better tradeoff between resource utilization and speed compared with the above existing designs.

7.3 FPGA Implementation of the Scheme

For the scheme implementation, we use a common FPGA design methodology of starting with a pure software implementation on the embedded processor, offloading the most time-consuming tasks to hardware in proper order and acquiring the design with a satisfactory tradeoff between area and speed. The polynomial operations, especially multiplication, is a hot topic for researchers on lattice-based cryptography, because it takes up a large part of computation in many LBC schemes, including Dilithium. In our implementation of signature, we use our polynomial operation components to achieve more than a half saving of time on our Cyclone platform and 39% on the Zynq platform. As for verification, also about a half of time is saved on Cyclone and the percentage saving is 18% for Zynq. The profiling results are included in Tables 4 and 5. Our second concern is the hash function, or Keccak to be exact, involved in Dilithium because it became the new bottleneck after we offloaded the polynomial multiplication/addition into hardware. For Cyclone, we get a second saving of about 70–80% for both signature and verification. But the improvements on the Zynq platform may be not be substantial, which is only around 3% considering the addition cost of area, so we try using NEON vector instructions to replace the original C code and acquire a more satisfactory improvement. After our measurement, we found that there is no considerable operation that is too time-consuming among the remaining software parts, so we do not design additional optimization techniques.

From the experiment results, in Cyclone, compared to the pure soft-core implementation on Nios II, the design including both polynomial operation and Keccak is approximately 11.2 times faster for signature and 7.4 times faster for verification. While on the Xilinx Zynq-7020, the built-in Cortex A9 processor, which has a great performance on its own and is driven by a high-frequency clock independent of the hardware modules, significantly improves the implementation speed of the C part. The improvement brought by offloading Keccak into hardware seems not very significant, so in the final design selection on Zynq we use the NEON version. Our polynomial hardware architecture still helped achieve 39% and 18% speed improvement for signature and verification, respectively, compared to the pure C implementation on Cortex A9. Lastly, the final improvements are 51.1% and 30.9% for signature and verification, respectively. In addition, we also provide two

Table 5. Verification Performance: A Comparative Summary

Verification	Environment	LUT	Time(ms)	Time Saving(%) ^{#1}	Speed-up ^{#2}
Our implementation ^I	Altera Cyclone IV EP4CE115F29C7	987	9.50×10^2	0	1
Our implementation ^{II}		3908	4.65×10^2	51.0	2.04
Our implementation ^{III}		8568	1.28×10^2	86.5	7.42
Our implementation ^I	Xilinx Zynq 7000 Soc XC7Z020-2CLG400I	576	1.44×10^1	0	1
Our implementation ^{II}		2620	1.18×10^1	18.0	1.22
Our implementation ^{III}		6253	1.15×10^1	20.1	1.25
Our implementation ^{IV}		2620	9.94	30.9	1.44
Implementation in [6]	TSMC 40nm LP CMOS	-	3.19	/	/
Implementation in [7] ^a	Xilinx Artix-7	108878	5.38×10^{-1}	0	1
Implementation in [7] ^b	/*	388991	5.38×10^{-1}	0.00	1.00

^{I,2,3}Setting respectively: naked Nios II, Nios II + polynomial(HW), Nios II + polynomial(HW) + Keccak(HW).

^{I,II,III,IV}Setting respectively: naked ARM Cortex-A9, ARM Cortex-A9 + polynomial(HW), ARM Cortex-A9 + polynomial(HW) + Keccak (HW), ARM Cortex-A9 + polynomial(HW) + Keccak(NEON instructions). ^{a,b,c}Setting respectively: non-optimization, with loop unrolling, with loop pipelining. *In the work of [7], the exact device name is not provided.

^{#1, #2} Performance evaluation in both time saving percent and multiple of speed-up are provided for clarity (under the same environment platform). The baseline value setting are 0 and 1 respectively.

existing works of the implementation of the Dilithium scheme. To our knowledge, there are two existing entire implementations of Dilithium; however, both of them are not FPGA implementation targeting compactness. So we present the results only as general references in Tables 4 and 5. One of the implementations is in [6] where the NTT is also well-designed and we have compared it with our polynomial architecture in the last subsection. But the results of their configuration crypto-processor Sapphire are obtained by ASIC rather than classical FPGA implementation. In contrast, our implementation takes longer but the same order of magnitude time for signature and verification as with Sapphire. Another existing implementation result is in [7], where hardware implementations based on the HLS method of Dilithium along with several NIST PQC candidates are introduced, which target only high speed but do not consider the resources utilization. In contrast, our co-design achieves a decent tradeoff between area and speed, which implies our designs have potential for embedded devices with limited area resources.

8 CONCLUSION

Polynomial multiplication and SHA-3 Keccak function are two of the most computationally intensive operations in the Crystals-Dilithium scheme. In this article, we proposed an improved NTT architecture, by building on the work of Roy et al. [40]. We also presented cooperative implementations of the Dilithium signature scheme in both Cyclone and Zynq environment at a low resource cost. We demonstrated that on our Altera DE2-115 platform, our implementation is about 11.2 times faster for signature and 7.4 times faster for verification than the corresponding pure Nios II software implementation. In addition, we also achieved 51% and 31% speed improvement for signature and verification compared to the pure C implementation on ZYNQ-7020 platform with a high-performance ARM processor. In the future, we intend to explore alternative optimized implementation architectures to achieve better performance.

REFERENCES

- [1] Carlos Aguilar-Melchor, Joris Barrier, Serge Guelton, Adrien Guinet, Marc-Olivier Killijian, and Tancrede Lepoint. 2016. NFLlib: NTT-based fast lattice library. In *Cryptographers' Track at the RSA Conference*. Springer, 341–356.
- [2] Miklós Ajtai. 1996. Generating hard instances of lattice problems. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*. 99–108.

- [3] Gorjan Alagic, Gorjan Alagic, Jacob Alperin-Sheriff, Daniel Apon, David Cooper, Quynh Dang, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, et al. 2020. *Status report on the second round of the NIST PQC standardization process*. U.S. Department of Commerce, National Institute of Standards and Technology.
- [4] Erdem Alkim, Hülya Evkan, Norman Lahr, Ruben Niederhagen, and Richard Petri. 2020. ISA extensions for finite field arithmetic-accelerating kyber and NewHope on RISC-V. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2020 (2020), 219–242.
- [5] Michael Baentsch. 2019. The Dilithium Implementation in pq-Crystals. Retrieved from <https://github.com/pq-crystals/dilithium>.
- [6] Utsav Banerjee, Tenzin S. Ukyab, and Anantha P. Chandrakasan. 2019. Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols (Extended Version). *IACR Cryptol. ePrint Arch.* 2019 (2019), 1140.
- [7] Kanad Basu, Deepraj Soni, Mohammed Nabeel, and Ramesh Karri. 2019. NIST post-quantum cryptography-A hardware evaluation study. *IACR Cryptology ePrint Archive* 2019 (2019), 47.
- [8] Günter Baszenski and Manfred Tasche. 1997. Fast polynomial multiplication and convolutions related to the discrete cosine transform. *Linear Algebra Appl.* 252, 1-3 (1997), 1–25.
- [9] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Trans. Comput. Theory (TOCT)* 6, 3 (2014), 1–36.
- [10] David G. Cantor and Erich Kaltofen. 1991. On fast multiplication of polynomials over arbitrary algebras. *Acta Inf.* 28, 7 (1991), 693–701.
- [11] Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray Perlner, and Daniel Smith-Tone. 2016. *Report on Post-Quantum Cryptography*. Vol. 12. U.S. Department of Commerce, National Institute of Standards and Technology.
- [12] James W. Cooley and John W. Tukey. 1965. An algorithm for the machine calculation of complex Fourier series. *Math. Comput.* 19, 90 (1965), 297–301.
- [13] Viet B. Dang, Farnoud Farahmand, Michal Andrzejczak, and Kris Gaj. 2019. Implementing and benchmarking three lattice-based post-quantum cryptography algorithms using software/hardware codesign. In *2019 International Conference on Field-Programmable Technology (ICFPT'19)*. 206–214.
- [14] Chaohui Du and Guoqiang Bai. 2016. Towards efficient polynomial multiplication for lattice-based cryptography. In *2016 IEEE International Symposium on Circuits and Systems (ISCAS'16)*. 1178–1181.
- [15] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2017. *Crystals-Dilithium*.
- [16] Morris J. Dworkin. 2015. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. Technical Report.
- [17] Xiang Feng, Shuguo Li, and Sufen Xu. 2019. RLWE-oriented high-speed polynomial multiplier utilizing multi-lane Stockham NTT algorithm. *IEEE Trans. Circ. Syst. II: Express Briefs* 67, 3 (2019), 556–559.
- [18] Tim Fritzmann, Georg Sigl, and Johanna Sepúlveda. 2020. RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography. *IACR Cryptol. ePrint Arch.* 2020 (2020), 446.
- [19] W. Morven Gentleman and Gordon Sande. 1966. Fast Fourier transforms: For fun and profit. In *Proceedings of the November 7–10, 1966, Fall Joint Computer Conference (AFIPS'66)*. 563–578.
- [20] Norman Göttert, Thomas Feller, Michael Schneider, Johannes Buchmann, and Sorin Huss. 2012. On the design of hardware building blocks for modern lattice-based encryption schemes. In *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 512–529.
- [21] David Harvey and Joris van der Hoeven. 2019. Faster polynomial multiplication over finite fields using cyclotomic coefficient rings. *Journal of Complexity* 54 (2019), 101404.
- [22] James Howe, Ciara Moore, Máire O'Neill, Francesco Regazzoni, Tim Güneysu, and Kevin Beeden. 2016. Lattice-based encryption over standard lattices in hardware. In *2016 53rd ACM/EDAC/IEEE Design Automation Conference (DAC'16)*. IEEE, 1–6.
- [23] Anatolii Alekseevich Karatsuba and Yu P. Ofman. 1962. Multiplication of many-digital numbers by automatic computers. In *Doklady Akademii Nauk*, Vol. 145. Russian Academy of Sciences, 293–294.
- [24] Po-Chun Kuo, Wen-Ding Li, Yu-Wei Chen, Yuan-Che Hsu, Bo-Yuan Peng, Chen-Mou Cheng, and Bo-Yin Yang. 2017. High performance post-quantum key exchange on FPGAs. *Cryptology ePrint Archive*.
- [25] Weiqiang Liu, Sailong Fan, Ayesha Khalid, Ciara Rafferty, and Máire O'Neill. 2019. Optimized schoolbook polynomial multiplication for compact lattice-based cryptography on FPGA. *IEEE Trans. VLSI Syst.* 27, 10 (2019), 2459–2463.
- [26] Vadim Lyubashevsky. 2012. Lattice signatures without trapdoors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 738–755.
- [27] Vadim Lyubashevsky and Daniele Micciancio. 2006. Generalized compact knapsacks are collision resistant. In *International Colloquium on Automata, Languages, and Programming*. Springer, 144–155.

- [28] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2013. On ideal lattices and learning with errors over rings. *J. ACM (JACM)* 60, 6 (2013), 1–35.
- [29] Ahmet Can Mert, Erdiñç Öztürk, and Erkey Savaş. 2019. Design and implementation of a fast and scalable NTT-based polynomial multiplier architecture. In *2019 22nd Euromicro Conference on Digital System Design (DSD'19)*. IEEE, 253–260.
- [30] Robert T. Moenck. 1976. Practical fast polynomial multiplication. In *Proceedings of the 3rd ACM Symposium on Symbolic and Algebraic Computation*. 136–148.
- [31] Peter L. Montgomery. 1985. Modular multiplication without trial division. *Math. Comput.* 44, 170 (1985), 519–521.
- [32] Karthikeyan Nagarajan, Sina Sayyah Ensan, Mohammad Nasim Imtiaz Khan, Swaroop Ghosh, and Anupam Chattopadhyay. 2019. SHINE: A novel SHA-3 implementation using ReRAM-based In-Memory computing. In *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED'19)*. IEEE, 1–6.
- [33] Hamid Nejatollahi, Nikil Dutt, Sandip Ray, Francesco Regazzoni, Indranil Banerjee, and Rosario Cammarota. 2017. Software and hardware implementation of lattice-based cryptography schemes. *University of California Irvine, CECS TR 17*, 4 (2017).
- [34] Tobias Oder and Tim Güneysu. 2017. Implementing the NewHope-Simple key exchange on low-cost FPGAs. In *International Conference on Cryptology and Information Security in Latin America*. Springer, 128–142.
- [35] Chris Peikert. 2009. Public-key cryptosystems from the worst-case shortest vector problem. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*. 333–342.
- [36] John M. Pollard. 1971. The fast Fourier transform in a finite field. *Math. Comput.* 25, 114 (1971), 365–374.
- [37] Thomas Pöppelmann, Léo Ducas, and Tim Güneysu. 2014. Enhanced lattice-based signatures on reconfigurable hardware. In *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 353–370.
- [38] Thomas Pöppelmann, Tobias Oder, and Tim Güneysu. 2015. High-performance ideal lattice-based cryptography on 8-bit ATmega microcontrollers. In *International Conference on Cryptology and Information Security in Latin America*. Springer, 346–365.
- [39] Oded Regev. 2009. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM (JACM)* 56, 6 (2009), 1–40.
- [40] Sujoy Sinha Roy, Frederik Vercauteren, Nele Mentens, Donald Donglong Chen, and Ingrid Verbauwhede. 2014. Compact ring-LWE cryptoprocessor. In *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 371–391.
- [41] Michael Schneider. 2013. Sieving for shortest vectors in ideal lattices. In *International Conference on Cryptology in Africa*. Springer, 375–391.
- [42] Gregor Seiler. 2018. Faster AVX2 optimized NTT multiplication for Ring-LWE lattice cryptography. *IACR Cryptology ePrint Archive* 2018 (2018), 39.
- [43] Peter W. Shor. 1999. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Rev.* 41, 2 (1999), 303–332.
- [44] Thom Wiggers. [n. d.]. PQClean: Clean, portable, tested implementations of post-quantum cryptography. Retrieved from <https://github.com/PQClean/PQClean>.
- [45] Ming Ming Wong, Jawad Haj-Yahya, Suman Sau, and Anupam Chattopadhyay. 2018. A new high throughput and area efficient SHA-3 implementation. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS'18)*. IEEE, 1–5.
- [46] Guozhu Xin, Jun Han, Tianyu Yin, Yuchao Zhou, Jianwei Yang, Xu Cheng, and Xiaoyang Zeng. 2020. VPQC: A domain-specific vector processor for post-quantum cryptography based on RISC-V architecture. *IEEE Trans. Circ. Syst. I-Regular Papers* 67, 8 (2020), 2672–2684.

Received June 2020; revised August 2020; accepted January 2021